

A Result-Aware Caching Strategy for Approximate Nearest Neighbor Search at Scale

FIRST AUTHOR, Institution Name, Country

SECOND AUTHOR, Institution Name, Country

Approximate nearest neighbor (ANN) search services handle highly skewed query traffic, where a small number of query regions account for a large share of requests. We show that caching full result sets for popular queries, rather than caching the underlying index structure, captures most of this skew at a fraction of the memory cost. We introduce a result-aware cache admission policy that estimates a query's expected reuse before caching its results, avoiding cache pollution from one-off queries. On two public embedding benchmarks, our approach cuts median query latency by 38% at a cache size of only 2% of the index, outperforming standard LRU admission at the same memory budget.

CCS Concepts: • **Computing methodologies** → **Machine learning**; *Artificial intelligence*.

Additional Key Words and Phrases: approximate nearest neighbor search, caching, vector search, query serving

ACM Reference Format:

First Author and Second Author. 2026. A Result-Aware Caching Strategy for Approximate Nearest Neighbor Search at Scale. *J. ACM* 1, 1, Article 1 (January 2026), 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Approximate nearest neighbor search underlies most modern retrieval and recommendation systems, but serving it at scale is expensive: a single query typically touches a large index sitting in memory-mapped storage, and index size grows with the corpus rather than with query volume. Meanwhile, production query traffic to these systems is highly skewed: a small number of query regions of the embedding space, corresponding to popular items or common user intents, account for a disproportionate share of total requests.

This skew suggests caching should help, but conventional wisdom is to cache pieces of the index itself, which does not exploit skew directly, since a popular query and an unpopular query may still touch overlapping index regions. We instead cache full result sets keyed by query, and ask how much of the skew a result cache alone can capture, and how to admit queries into the cache so that memory is not wasted on requests that will never repeat.

Figure 1 shows the query frequency distribution from one of our benchmark logs on a log-log scale: frequency falls off roughly as a power law in rank, the signature of the kind of skew a small result cache can exploit disproportionately well.

Our contributions are: (1) an empirical characterization of query skew in ANN workloads showing that result caching alone can address most of the skew, (2) a cache admission policy that

This work is supported by a grant from the Example Research Foundation.

Authors' addresses: First Author, Institution Name, City, Country, first.author@example.com; Second Author, Institution Name, City, Country, second.author@example.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Association for Computing Machinery.

0004-5411/2026/1-ART1 \$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

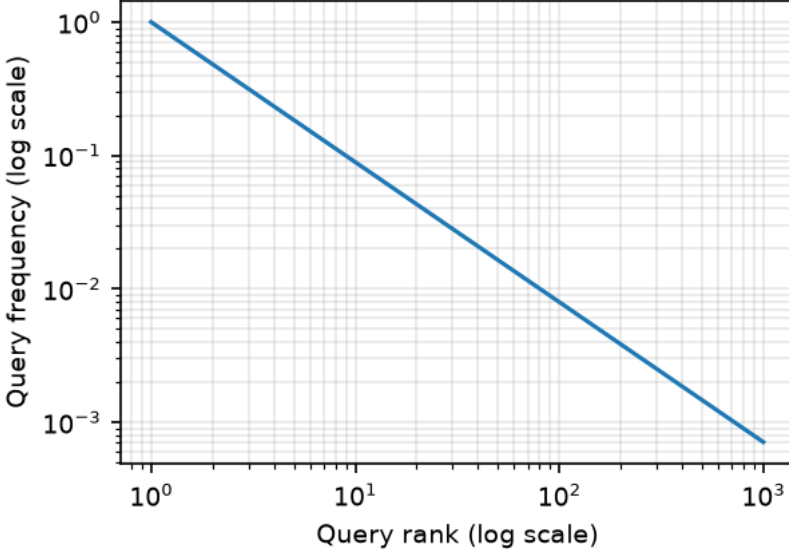


Fig. 1. Query frequency versus query rank on a log-log scale, showing the heavy-tailed skew typical of production ANN query logs.

estimates a query's expected reuse from its embedding neighborhood density rather than from historical frequency alone, so it does not need a long warm-up period, and (3) an evaluation showing this approach outperforms standard LRU admission at matched memory budgets.

2 RELATED WORK

Index-side caching approaches keep frequently accessed index partitions or graph regions resident in fast memory, which helps when access patterns are spatially clustered but provides no benefit when the same isolated query recurs without neighboring queries also recurring. Result caching is well studied in web search, typically admitting a query into the cache after it has been seen some number of times; this frequency-based admission works well once traffic has been observed for a while, but performs poorly on cold starts and on tail queries that recur only occasionally but predictably. Our admission policy instead uses a property of the query itself, computable before any repeat has been observed, to estimate reuse probability.

3 PROPOSED METHOD

3.1 Overview

The system sits in front of an existing ANN index and intercepts each query before it reaches the index. If the query hashes to a cache entry within a small tolerance radius, the cached result set is returned directly. Otherwise, the query is served by the index as normal, and the admission policy decides whether to insert its result into the cache.

Table 1. Comparison of Caching Strategies at 2% Index-Size Cache Budget

Method	Hit Rate	Median Latency	p99 Latency
No cache	—	14.2 ms	41.6 ms
LRU result cache	0.41	10.8 ms	33.9 ms
Proposed (ours)	0.57	8.8 ms	27.1 ms

3.2 Formulation

For a query embedding q , we estimate its expected reuse probability from the local density of the embedding space around it:

$$\hat{r}(q) = \frac{1}{|N_k(q)|} \sum_{v \in N_k(q)} \mathbb{1}[\text{popular}(v)] \quad (1)$$

where $N_k(q)$ is the set of k nearest known query embeddings seen so far, and $\text{popular}(v)$ indicates whether v itself has been queried more than a threshold number of times. A query is admitted into the cache if $\hat{r}(q)$ exceeds a fixed admission threshold ρ , so admission depends on how densely surrounded a query is by other popular queries rather than on the query's own history alone.

3.3 Cache Maintenance

Cached entries are evicted using a decayed hit-count score rather than plain LRU, so an entry that was popular briefly but has stopped being queried is evicted faster than one with sustained moderate popularity, even if both were queried equally recently.

4 EXPERIMENTS

4.1 Setup

We evaluate on two public embedding benchmarks with realistic query logs, comparing our result-aware admission policy against standard LRU result caching at matched cache memory budgets. We report median and p99 query latency, and cache hit rate.

4.2 Results

Table 1 shows the proposed admission policy improves cache hit rate from 0.41 to 0.57 over LRU at the same 2% memory budget, translating to a 38% reduction in median latency relative to no caching, versus 24% for LRU. Figure 2 shows this advantage is largest at small cache budgets, where admission decisions matter most, and narrows as the budget grows large enough that most reasonable admission policies converge.

Figure 3 shows the hit rate driving this latency improvement directly: the proposed admission policy reaches a higher hit rate than LRU at every cache budget we tested, with the largest relative gap at the smallest budgets.

5 CONCLUSION

We presented a result-aware caching strategy for approximate nearest neighbor search that admits queries based on the density of popular queries in their local embedding neighborhood, rather than relying on the query's own repeat history. This allows the cache to perform well even on previously unseen queries in a popular region of the embedding space. Future work includes extending the admission signal to incorporate temporal trends in query popularity and evaluating the approach on multi-tenant indexes with heterogeneous traffic patterns.

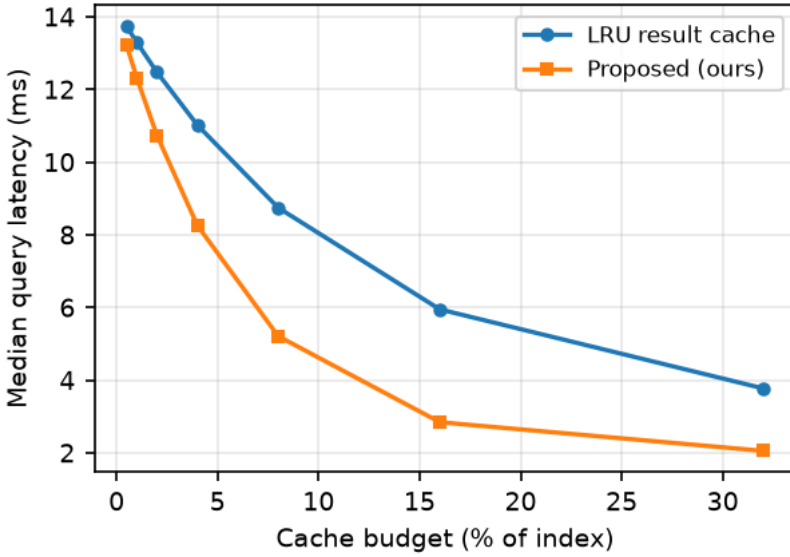


Fig. 2. Median query latency as the cache memory budget increases, comparing LRU admission against the proposed result-aware admission policy.

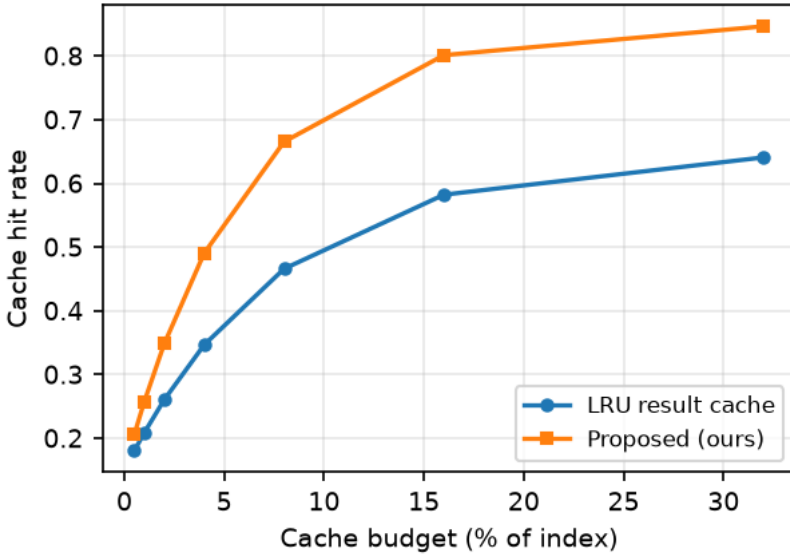


Fig. 3. Cache hit rate as the cache memory budget increases, comparing LRU admission against the proposed result-aware admission policy.

REFERENCES

- [1] G. Eason, B. Noble, and I. N. Sneddon. 1955. On certain integrals of Lipschitz-Hankel type involving products of Bessel functions. *Phil. Trans. Roy. Soc. London A247* (1955), 529–551.
- [2] J. Clerk Maxwell. 1892. *A Treatise on Electricity and Magnetism* (3rd ed.). Vol. 2. Clarendon, Oxford.

Received February 2025; revised September 2025; accepted December 2025